

Short Design Report: Concurrent Task Dispatcher in Rust

1. Overview

This project implements a concurrent task dispatcher in Rust. The system models a stream of incoming CPU-bound and IO-bound tasks, places them into queues, assigns them to a fixed worker pool, and records metrics about performance and fairness. The design follows the project requirement that the system be genuinely concurrent and queue-based rather than just a loop that processes jobs one at a time.

The main goal was to make the design easy to explain during a demo while still showing real scheduling decisions. For that reason, the final system uses a generator thread, dispatcher logic, worker threads, and a monitor thread. This keeps responsibilities separated and makes it easy to describe where a task is, who owns it, and what happens next.

2. Architecture

The project uses four main components:

1. Task generator

- Creates tasks using a fixed random seed
- Mixes CPU and IO tasks
- Simulates staggered arrivals over time
- Sends tasks into the dispatcher through a channel

2. Dispatcher

- Receives tasks from the generator
- Stores them in one queue or two queues depending on policy
- Tracks which workers are free
- Checks whether dispatching the next task would exceed the global 100% CPU budget
- Sends the chosen task to a worker

3. Worker pool

- Fixed size of 10 workers
- Each worker waits on its personal bounded channel
- Simulates task execution with sleep(duration)
- Records completion and becomes available again

4. Monitor

- Samples queue length, number of busy workers, and current simulated CPU usage every 10 ms
- Provides data for average queue length, peak queue length, and average CPU usage

This architecture follows the central dispatcher style: generator -> dispatcher -> workers, with monitoring added for instrumentation.

3. Data Structures and Synchronization

Core data structures

- Task
- id
- arrival_offset
- kind (CPU or IO)
- duration
- cpu_cost
- VecDeque
- Used for FIFO queueing
- Also used for the separate CPU and IO queues in the optimized policy
- SharedState
- total queue length
- CPU queue length
- IO queue length
- busy worker count
- current CPU load
- shutdown flag
- CollectorState
- run start time
- per-worker busy time
- completed task records

Synchronization strategy

The project deliberately uses both **channels** and **shared state** because they solve different problems.

Channels

Channels are used for ownership transfer and event flow:

- generator -> dispatcher: new tasks entering the system
- workers -> dispatcher: worker availability notifications
- dispatcher -> workers: actual task assignment

This keeps task movement explicit and avoids accidental data races on task ownership.

Arc>

Shared state is used for system-wide metrics and monitoring:

- queue lengths
- busy worker count
- CPU load

- worker busy time
- completed task records

This state needs to be visible to multiple threads at once, so shared ownership plus locking is appropriate.

4. Scheduling Policies

FIFO policy

The FIFO policy uses one shared ready queue. The dispatcher only looks at the front task. If a worker is available and the task fits inside the remaining simulated CPU budget, it is dispatched.

What improved:

- very simple logic
- easy to explain and defend
- direct baseline for comparison

What became worse:

- head-of-line blocking can happen
- one CPU-heavy task can delay many smaller IO tasks
- fairness between task classes can degrade when the workload is stressed

Optimized policy

The optimized policy uses two queues: one for CPU tasks and one for IO tasks.

Behavior:

- two workers are reserved to prefer CPU tasks first
- non-reserved workers prefer IO first
- if the preferred queue cannot dispatch because of CPU budget or emptiness, the dispatcher tries the other queue

What improved:

- lighter IO tasks can keep flowing even under CPU-heavy conditions
- queue buildup is reduced in stressed runs
- turnaround tends to improve when CPU tasks dominate

What became worse or more complicated:

- the policy is less intuitive than FIFO
- worker reservation introduces more decision logic
- starvation is reduced but not completely impossible under extreme patterns

5. Metrics Collected

The implementation records:

- total tasks completed

- makespan
- average wait time
- average turnaround time
- max wait time
- average simulated CPU usage
- average busy workers
- max queue length
- average queue length
- worker utilization
- number of CPU tasks completed
- number of IO tasks completed
- fairness gap between CPU and IO average wait time

These metrics give both performance and fairness visibility. Makespan and turnaround show throughput. Queue length and CPU usage show pressure. Fairness gap highlights whether one task class is being delayed much more than the other.

6. Experiments

Experiment A: Balanced workload

This run models a mostly mixed system:

- 1000 tasks
- roughly 70% IO and 30% CPU
- arrivals centered near 20 ms
- CPU and IO durations around 200 ms

The purpose is to show baseline scheduler behavior when the workload is not especially hostile.

Experiment B: Stressed workload

This run is designed to make the scheduler struggle:

- 1000 tasks
- more CPU-heavy work
- burstier arrivals
- longer CPU durations

The purpose is to expose the weakness of simple FIFO and to show why queue separation and class-aware dispatch can help.

7. Actual Results From Program Runs

Balanced workload - FIFO

- total tasks completed: 1000
- makespan: 41.151 s
- average wait time: 10841.829 ms
- average turnaround time: 11042.464 ms
- max wait time: 21299.875 ms
- average CPU usage: 90.48%
- average queue length: 263.18
- max queue length: 519
- worker utilization: about 47.70% to 50.26% across workers
- fairness gap: 119.771 ms

Balanced workload - Optimized

- total tasks completed: 1000
- makespan: 43.204 s
- average wait time: 5086.873 ms
- average turnaround time: 5287.383 ms
- max wait time: 23470.146 ms
- average CPU usage: 86.21%
- average queue length: 117.58
- max queue length: 232
- worker utilization: about 45.87% to 46.82% across workers
- fairness gap: 17400.888 ms

Stressed workload - FIFO

- total tasks completed: 1000
- makespan: 105.900 s
- average wait time: 48343.290 ms
- average turnaround time: 48611.667 ms
- max wait time: 97209.764 ms
- average CPU usage: 84.85%
- average queue length: 456.66
- max queue length: 918
- worker utilization: about 23.95% to 26.09% across workers
- fairness gap: 950.167 ms

Stressed workload - Optimized

- total tasks completed: 1000
- makespan: 110.799 s
- average wait time: 36533.748 ms

- average turnaround time: 36802.233 ms
- max wait time: 102117.460 ms
- average CPU usage: 81.13%
- average queue length: 329.70
- max queue length: 663
- worker utilization: about 23.94% to 24.44% across workers
- fairness gap: 53684.298 ms

8. Result Interpretation

In the balanced workload, the optimized policy reduced average wait time from 10841.829 ms to 5086.873 ms and reduced average turnaround time from 11042.464 ms to 5287.383 ms. It also lowered the average and maximum queue lengths. However, its makespan was slightly worse at 43.204 s compared to 41.151 s, and its fairness gap became much larger. This means the optimized scheduler improved overall responsiveness, but it did so less evenly across CPU and IO tasks.

In the stressed workload, the optimized policy again reduced average wait time from 48343.290 ms to 36533.748 ms and reduced average turnaround time from 48611.667 ms to 36802.233 ms. It also lowered queue buildup, reducing the average queue length from 456.66 to 329.70 and the maximum queue length from 918 to 663. However, makespan increased from 105.900 s to 110.799 s and fairness became much worse, with the fairness gap growing sharply. This shows that the optimized scheduler handled congestion better, but the queue preferences and CPU reservation strategy created a stronger imbalance between task classes.

9. Bugs, Mistakes, and Trade-offs

A common risk in this kind of program is writing something that looks concurrent but is effectively sequential. To avoid that, the design separates responsibilities clearly: generation, dispatch, execution, and monitoring all happen in different threads. Another risk is shutdown bugs, where workers or monitors wait forever. This implementation uses explicit Shutdown messages for workers and a shared done flag for the monitor so that the program can terminate cleanly.

Another important trade-off is lock granularity. A single giant lock around all queues and metrics would have been easier to write at first, but it would make the system harder to reason about and more serialized. The final design uses channels for task movement and keeps shared locks mostly for metrics and lightweight state.

10. Remaining Fairness Risks

Fairness is improved in the optimized design, but starvation is still theoretically possible if CPU-heavy jobs keep arriving at the wrong times relative to available CPU budget. The optimized scheduler helps by separating queues and reserving some CPU preference, but it does not implement full aging. Adding an aging system would be a reasonable extension if stronger anti-starvation behavior were needed.

11. Lessons Learned

The main lesson from the project is that a scheduler is not just a queue plus threads. The interesting part is the policy: what to send next, under what constraints, and what trade-offs that creates. Separating task arrival, queueing, dispatch, execution, completion, and monitoring made the system easier to debug and easier to explain.